Big O notation

- **print_numbers_times_2**

```python
def print_numbers_times_2(numbers_list):
  for number in numbers_list:
    print(number * 2)
```

En este caso tenemos un ciclo que repetirá dependiendo del tamaño de una lista, por lo tanto el tiempo es O(n).

- **check_if_lists_have_an_equal**

```python
def check_if_lists_have_an_equal(list_a, list_b):
  for element_a in list_a:
    for element_b in list_b:
      if element_a == element_b:
        return True

  return False
```

En este caso el tiempo depende de una lista anidada en otra lista, por lo tanto es igual a O(n^2).

- **print_10_or_less_elements**

```python
def print_10_or_less_elements(list_to_print):
  list_len = len(list_to_print)
  for index in range(min(list_len, 10)):
    print(list_to_print[index])
```

Aqui tenemos, primero, una instruccion que es tiempo constante O(1) y luego un ciclo que se ejecutará como maximo 10 veces sin importar el tamaño de la entrada, por lo que es tambien tiempo constante O(1). El resultado es O(1)

- **generate_list_trios**

```python
def generate_list_trios(list_a, list_b, list_c):
  result_list = []
  for element_a in list_a:
    for element_b in list_b:
      for element_c in list_c:
        result_list.append(f'{element_a} {element_b} {element_c}')

  return result_list
```

Mismo caso que el segundo pero en tres loops anidados, por lo tanto es O(n^3)

Big O notation

```python
def bubble_sort(list_to_sort):
    # Repetimos la iteración de la lista por todos los elementos para moverlos al final
    for outer_index in range(0, len(list_to_sort) - 1):
        # Usamos esta variable para revisar si hemos movido elementos
        has_made_changes = False
        # Le restamos uno al length para parar en el penultimo elemento
        # Usamos el indice exterior para restar las ejecuciones de
        # los elementos que ya estan ordenados al final
        for index in range(0, len(list_to_sort) - 1 - outer_index):
            # Guardamos los valores del elemento actual y el siguiente
            current_element = list_to_sort[index]
            next_element = list_to_sort[index + 1]

            print(f'-- Iteracion {outer_index}, {index}. Elemento actual: {current_element}, Siguiente elemento: {next_element}')

            # Si el actual es mayor al siguiente, intercambiamos sus posiciones
            if current_element > next_element:
                print('El elemento actual es mayor al siguiente. Intercambiandolos...')
                list_to_sort[index] = next_element
                list_to_sort[index + 1] = current_element
                has_made_changes = True

        # Si no hemos movido elementos, la lista ya esta ordenada
        if not has_made_changes:
            return
```

Este codigo pertenece al material del tema de algorimos de ordenamiento.

En el algoritmo de bubble sort vamos a empezar con un loop que depende del tamaño de la lista a ordenar, por lo tanto esto es O(n), luego tenemos un loop interior en la linea 9 que decrese conforme avanzamos en el loop externo, esto sería un tiempo de O(n) tambien, por lo tanto el resultado es O(n^2)